LAPORAN PRAKTIKUM

POSTTEST 5

ALGORITMA PEMROGRAMAN LANJUT

Disusun oleh:

Denny Mulia 2509106011

Kelas A1‘25

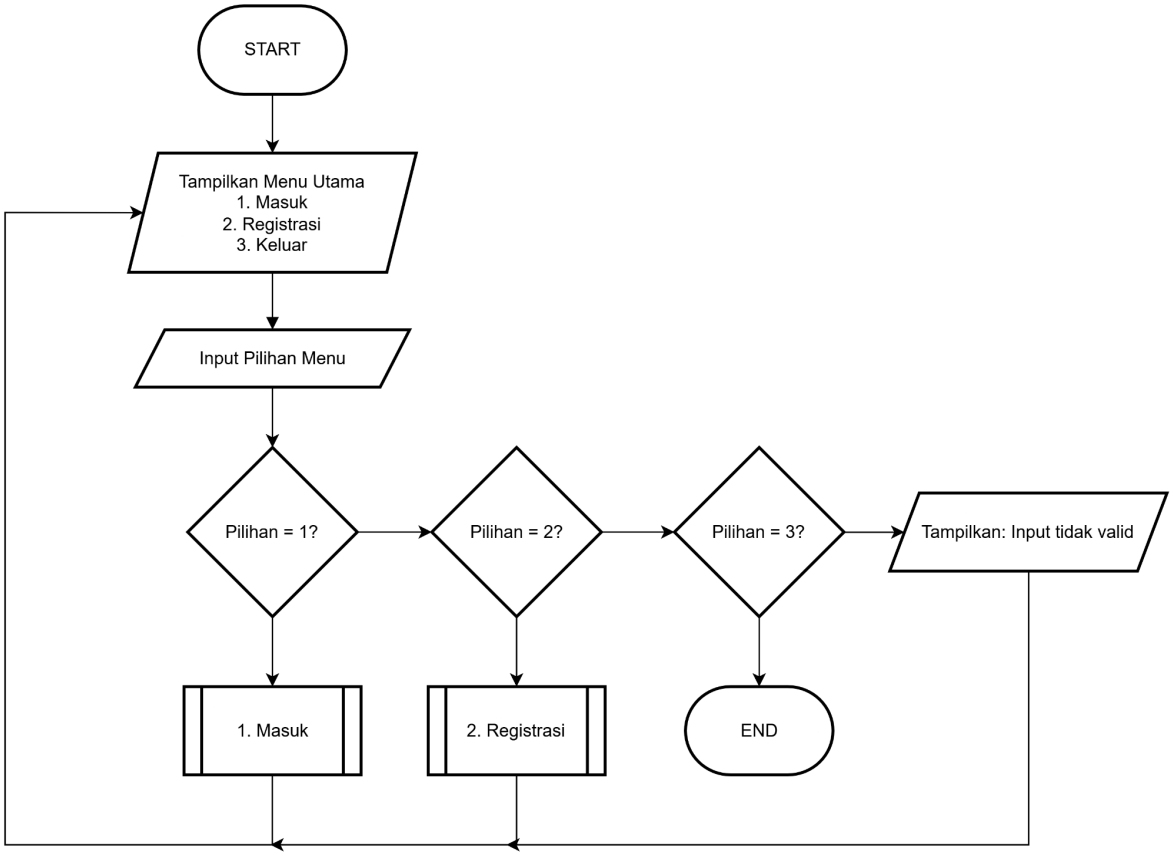
PROGRAM STUDI INFORMATIKA

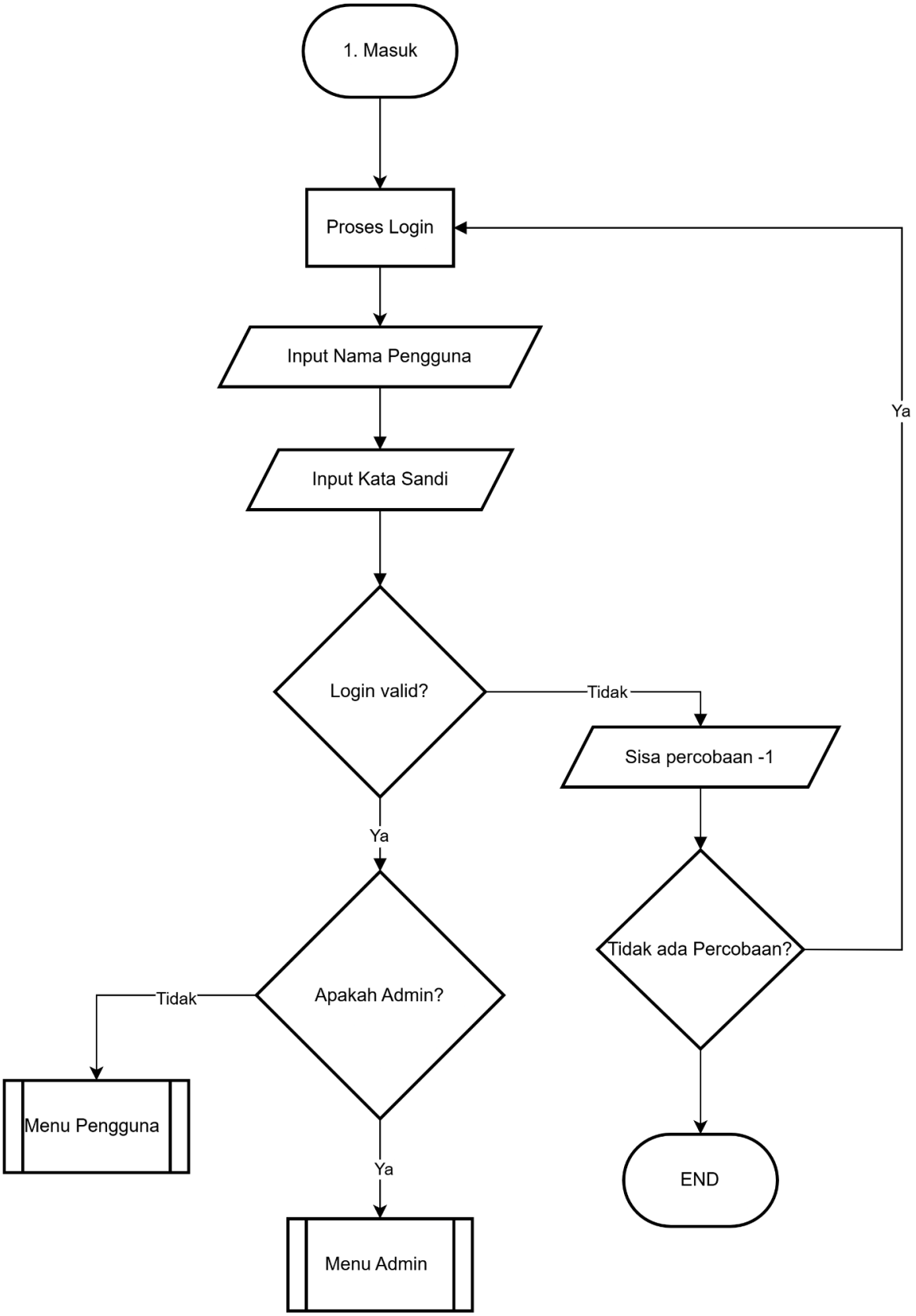
UNIVERSITAS MULAWARMAN

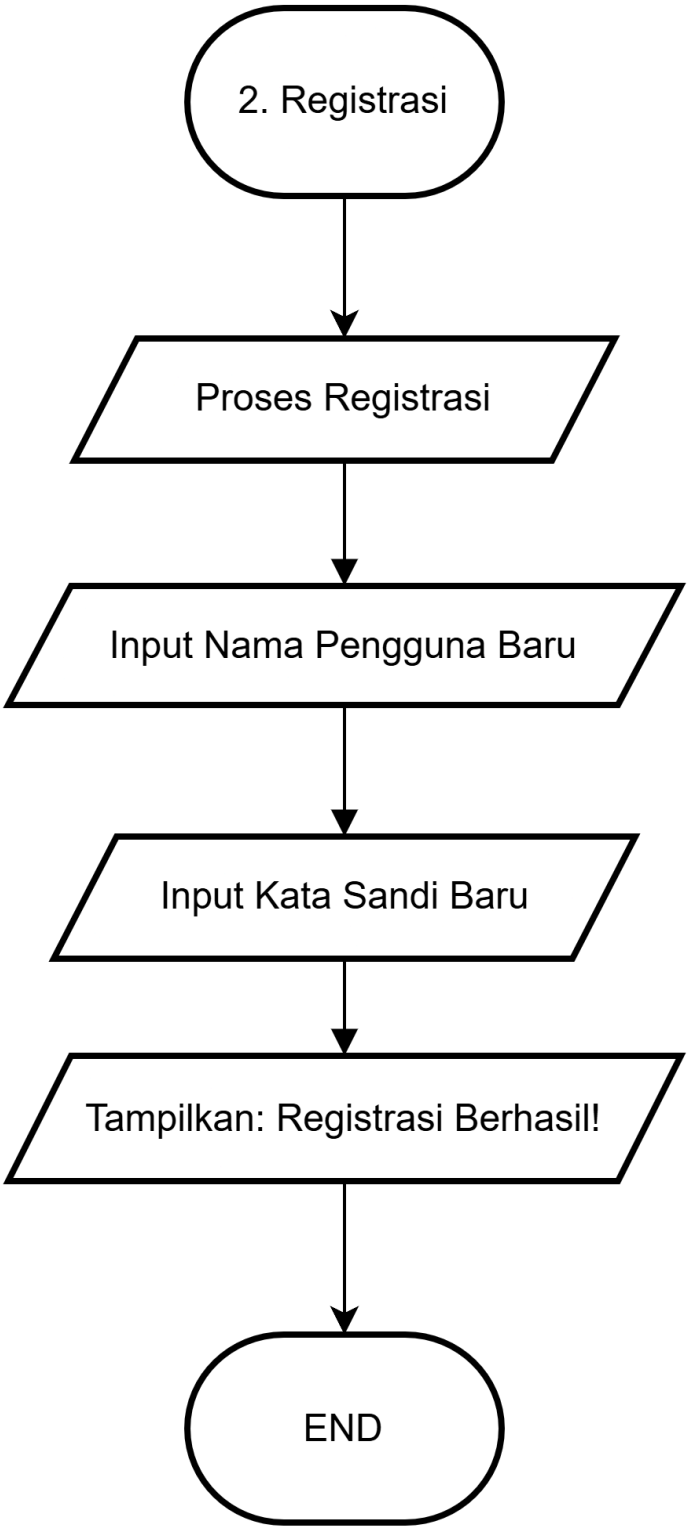
SAMARINDA

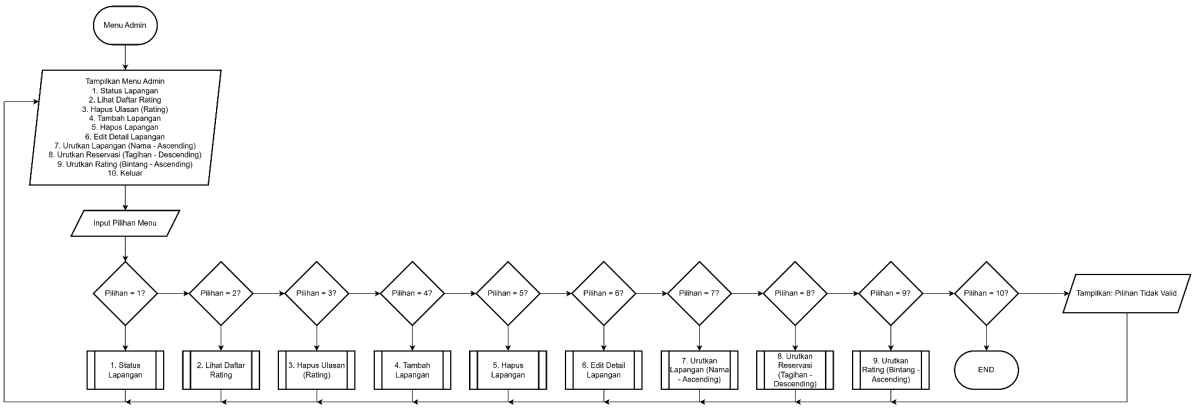
2025

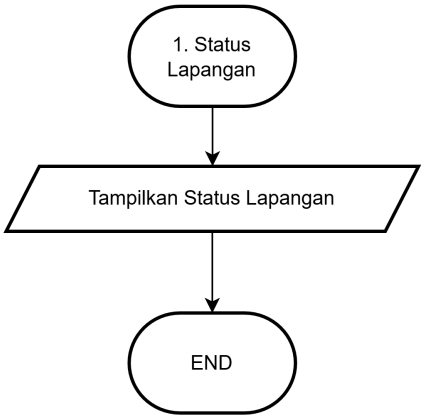
1. Flowchart

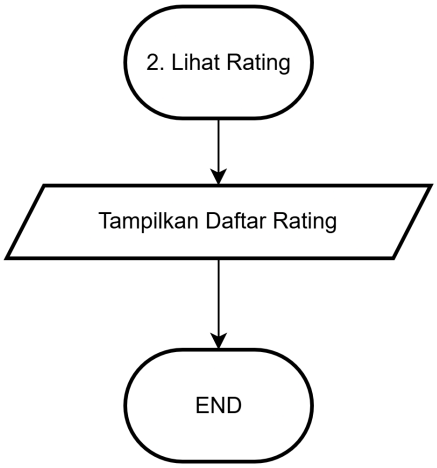

START
Tampilkan Menu Utama
1. Masuk
2. Registrasi
3. Keluar
Input Pilihan Menu
Pilihan = 1?
Pilihan = 2?
Pilihan = 3?
Tampilkan: Input tidak valid
1. Masuk
2. Registrasi
END

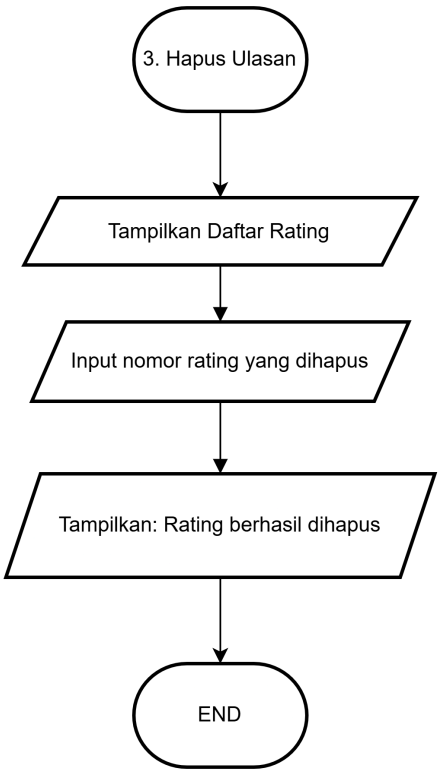

1. Masuk
Proses Login
Input Nama Pengguna
Input Kata Sandi
Login valid?
Tidak
Sisa percobaan -1
Ya
Tidak ada Percobaan?
Ya
Apakah Admin?
Tidak
Menu Pengguna
Ya
Menu Admin
END

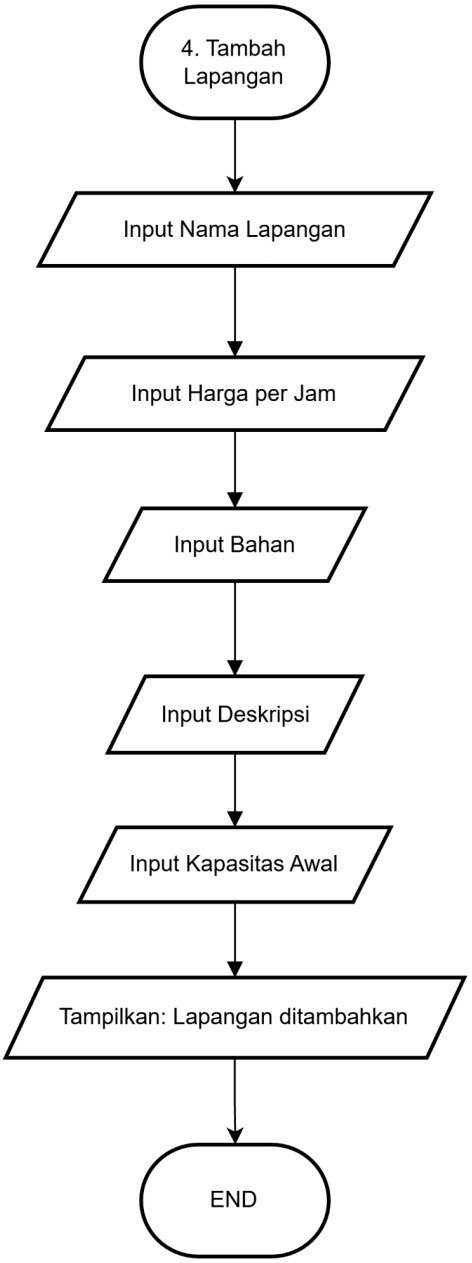

2. Registrasi
Proses Registrasi
Input Nama Pengguna Baru
Input Kata Sandi Baru
Tampilkan: Registrasi Berhasil!
END

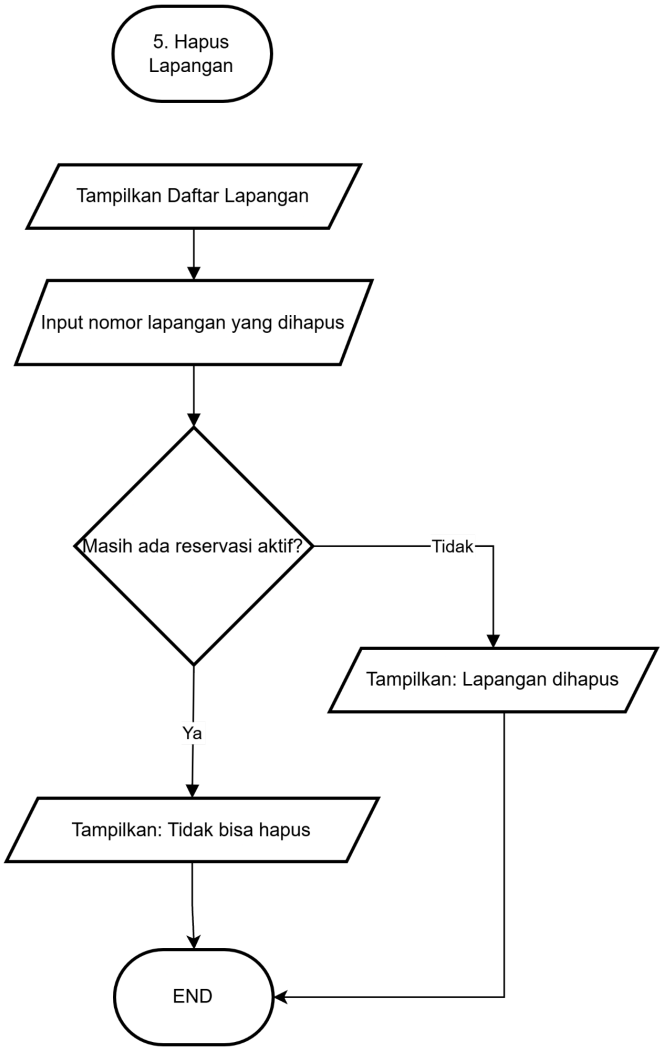

Menu Admin
Tampilkan Menu Admin
1. Status Lapangan
2. Lihat Daftar Rating
3. Hapus Ulasan (Rating)
4. Tambah Lapangan
5. Hapus Lapangan
6. Edit Detail Lapangan
7. Urutkan Lapangan (Nama - Ascending)
8. Urutkan Reservasi (Tagihan - Descending)
9. Urutkan Rating (Bintang - Ascending)
10. Keluar
Input Pilihan Menu
Pilihan = 1?
Pilihan = 2?
Pilihan = 3?
Pilihan = 4?
Pilihan = 5?
Pilihan = 6?
Pilihan = 7?
Pilihan = 8?
Pilihan = 9?
Pilihan = 10?
Tampilkan: Pilihan Tidak Valid
1. Status Lapangan
2. Lihat Daftar Rating
3. Hapus Ulasan (Rating)
4. Tambah Lapangan
5. Hapus Lapangan
6. Edit Detail Lapangan
7. Urutkan Lapangan (Nama - Ascending)
8. Urutkan Reservasi (Tagihan - Descending)
9. Urutkan Rating (Bintang - Ascending)
END

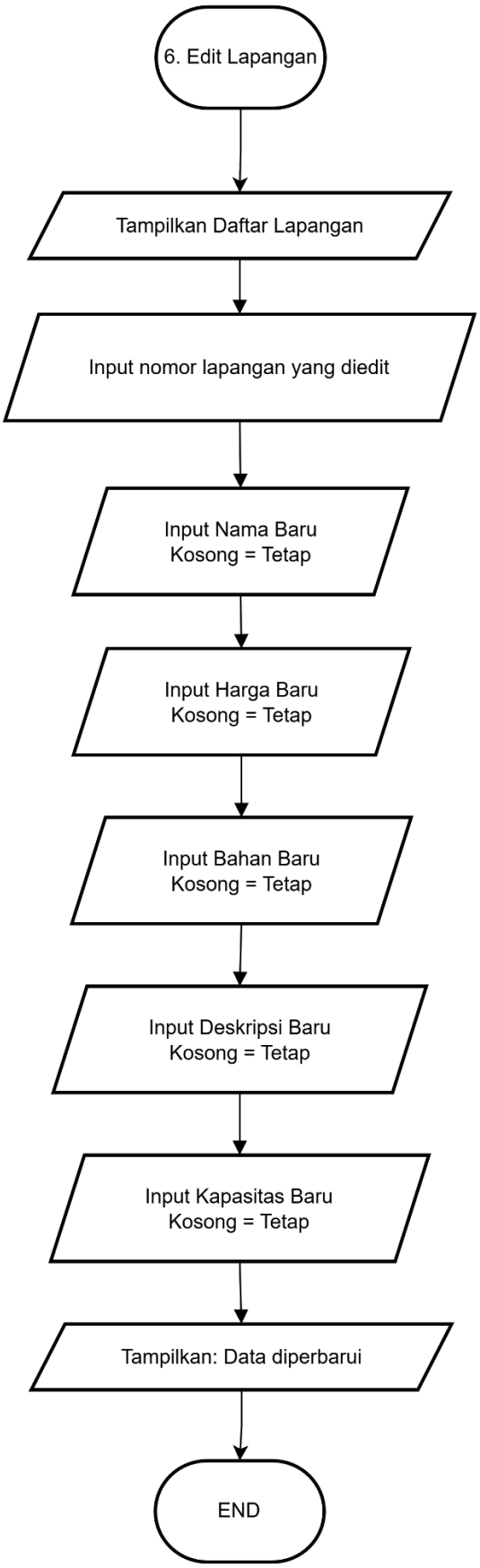

1. Status Lapangan
Tampilkan Status Lapangan
END

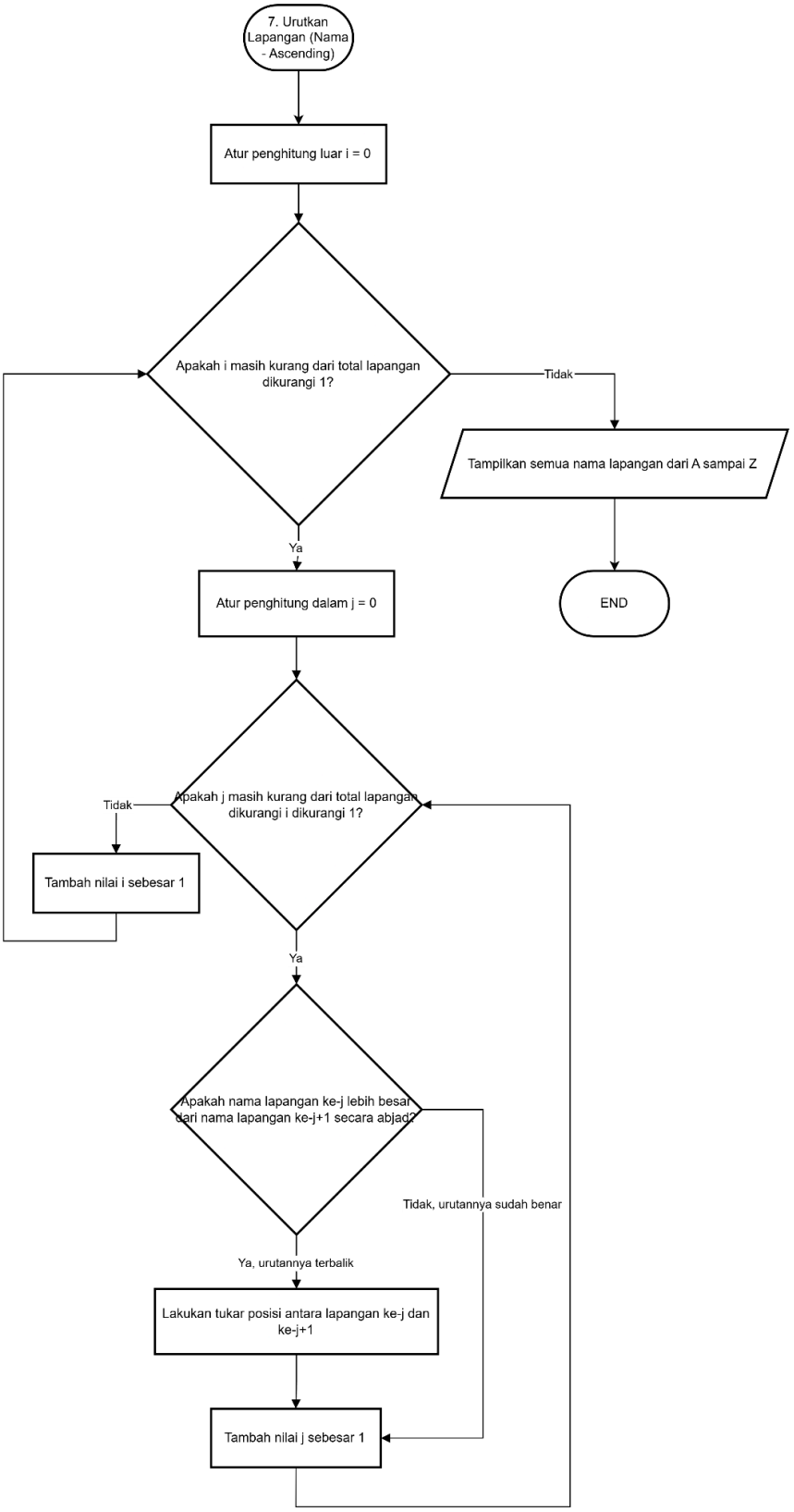

2. Lihat Rating
Tampilkan Daftar Rating
END

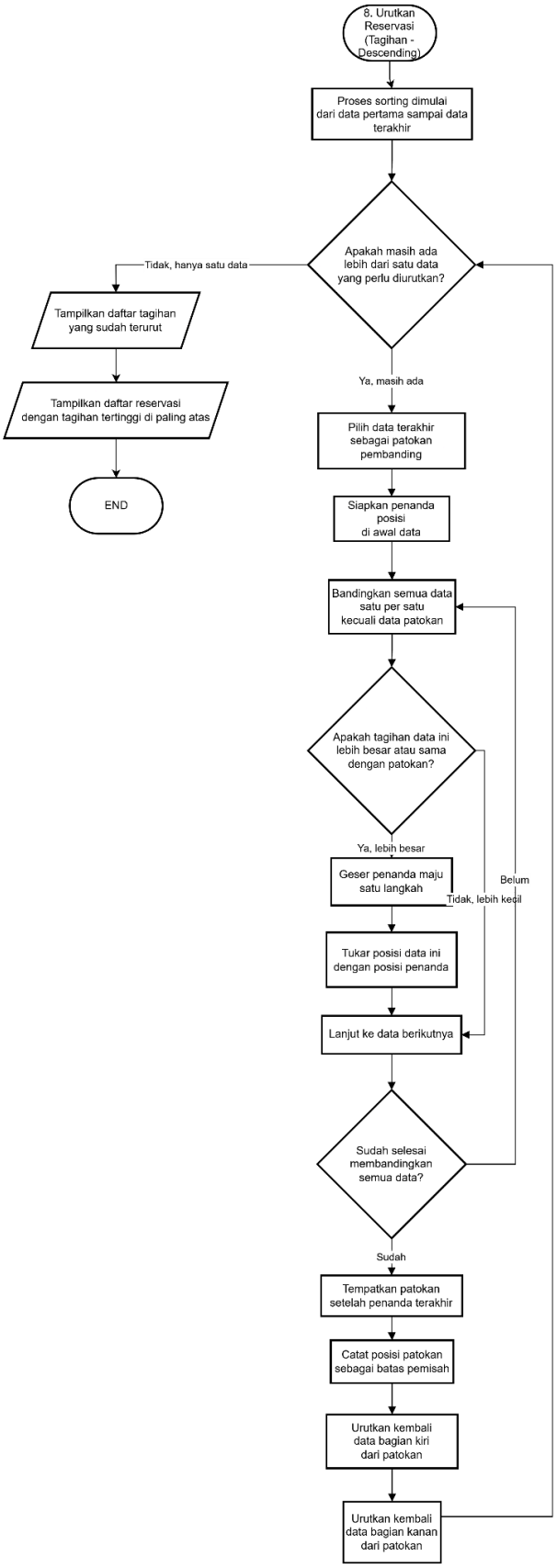

3. Hapus Ulasan
Tampilkan Daftar Rating
Input nomor rating yang dihapus
Tampilkan: Rating berhasil dihapus
END

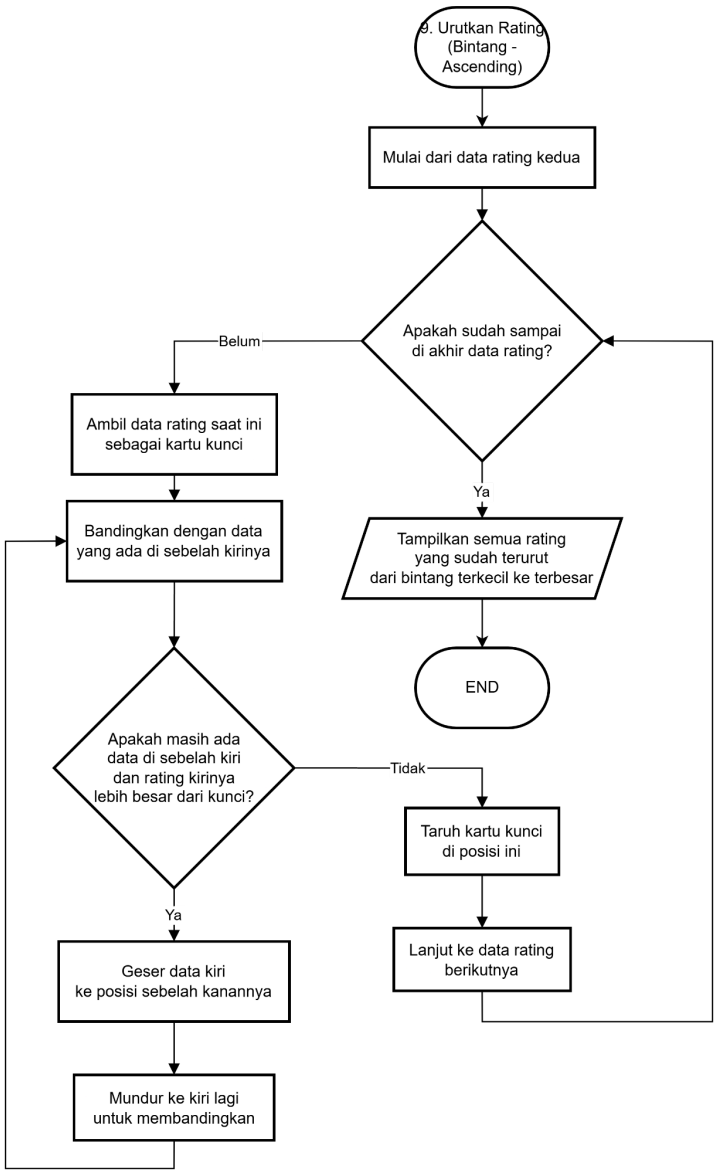

4. Tambah Lapangan
Input Nama Lapangan
Input Harga per Jam
Input Bahan
Input Deskripsi
Input Kapasitas Awal
Tampilkan: Lapangan ditambahkan
END

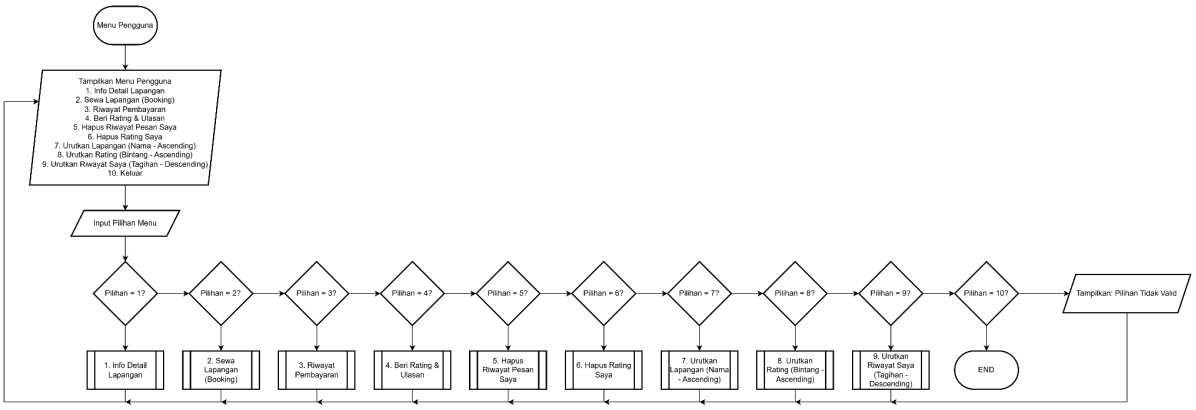

5. Hapus Lapangan
Tampilkan Daftar Lapangan
Input nomor lapangan yang dihapus
Masih ada reservasi aktif?
Tidak
Tampilkan: Lapangan dihapus
Ya
Tampilkan: Tidak bisa hapus
END

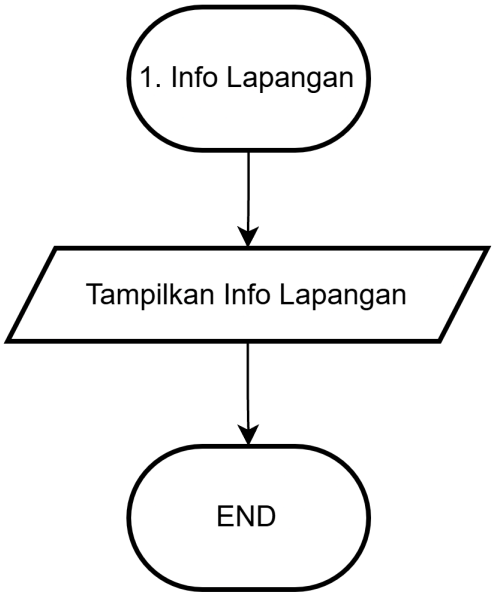

6. Edit Lapangan
Tampilkan Daftar Lapangan
Input nomor lapangan yang diedit
Input Nama Baru
Kosong = Tetap
Input Harga Baru
Kosong = Tetap
Input Bahan Baru
Kosong = Tetap
Input Deskripsi Baru
Kosong = Tetap
Input Kapasitas Baru
Kosong = Tetap
Tampilkan: Data diperbarui
END

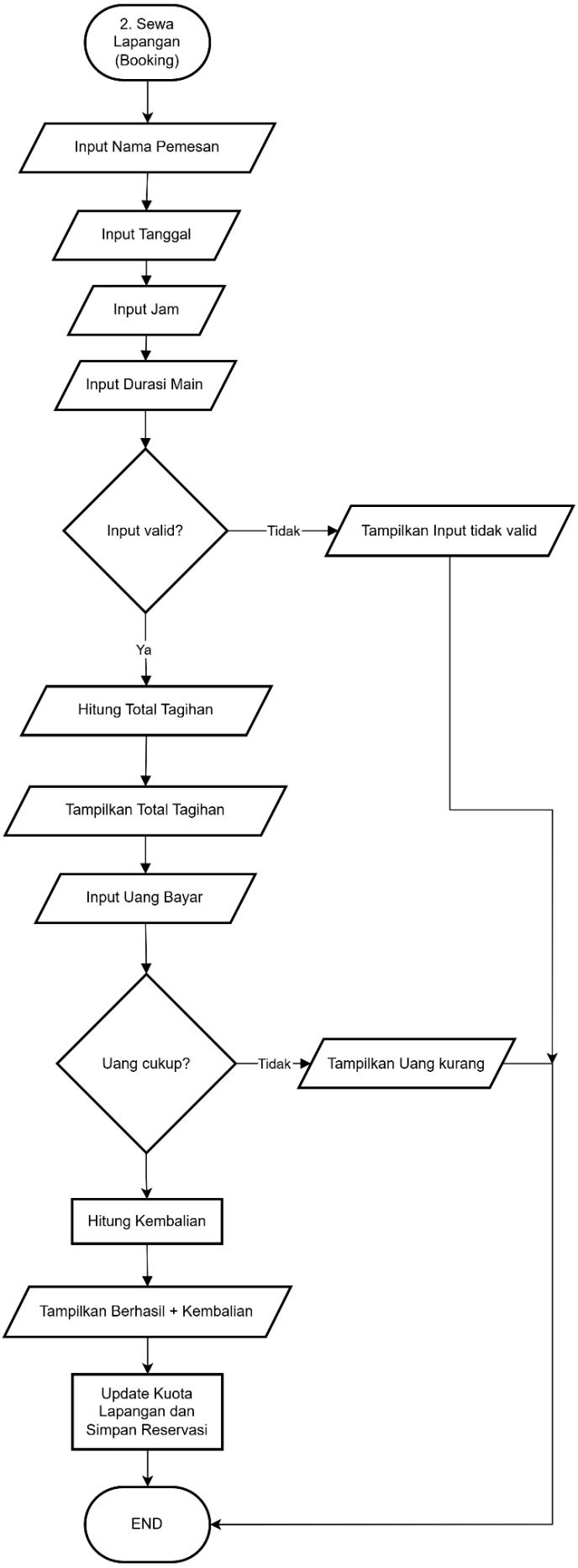

7. Urutkan Lapangan (Nama - Ascending)
Atur penghitung luar i = 0
Apakah i masih kurang dari total lapangan dikurangi 1?
Tidak
Tampilkan semua nama lapangan dari A sampai Z
END
Ya
Atur penghitung dalam j = 0
Apakah j masih kurang dari total lapangan dikurangi i dikurangi 1?
Tidak
Tambah nilai i sebesar 1
Ya
Apakah nama lapangan ke-j lebih besar dari nama lapangan ke-j+1 secara abjad?
Tidak, urutannya sudah benar
Ya, urutannya terbalik
Lakukan tukar posisi antara lapangan ke-j dan ke-j+1
Tambah nilai j sebesar 1

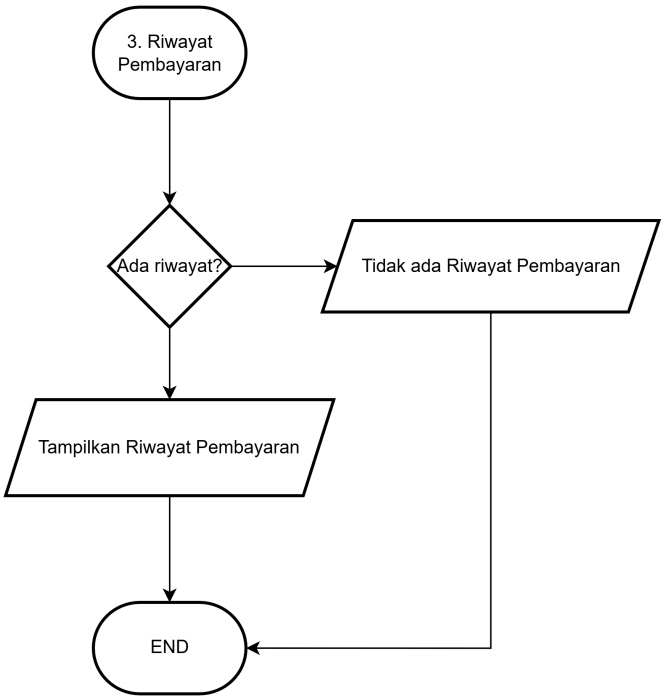

8. Urutkan Reservasi (Tagihan - Descending)
Proses sorting dimulai dari data pertama sampai data terakhir
Apakah masih ada lebih dari satu data yang perlu diurutkan?
Tidak, hanya satu data
Tampilkan daftar tagihan yang sudah terurut
Tampilkan daftar reservasi dengan tagihan tertinggi di paling atas
END
Ya, masih ada
Pilih data terakhir sebagai patokan pembanding
Siapkan penanda posisi di awal data
Bandingkan semua data satu per satu kecuali data patokan
Apakah tagihan data ini lebih besar atau sama dengan patokan?
Ya, lebih besar
Geser penanda maju satu langkah
Tidak, lebih kecil
Belum
Tukar posisi data ini dengan posisi penanda
Lanjut ke data berikutnya
Sudah selesai membandingkan semua data?
Sudah
Tempatkan patokan setelah penanda terakhir
Catat posisi patokan sebagai batas pemisah
Urutkan kembali data bagian kiri dari patokan
Urutkan kembali data bagian kanan dari patokan

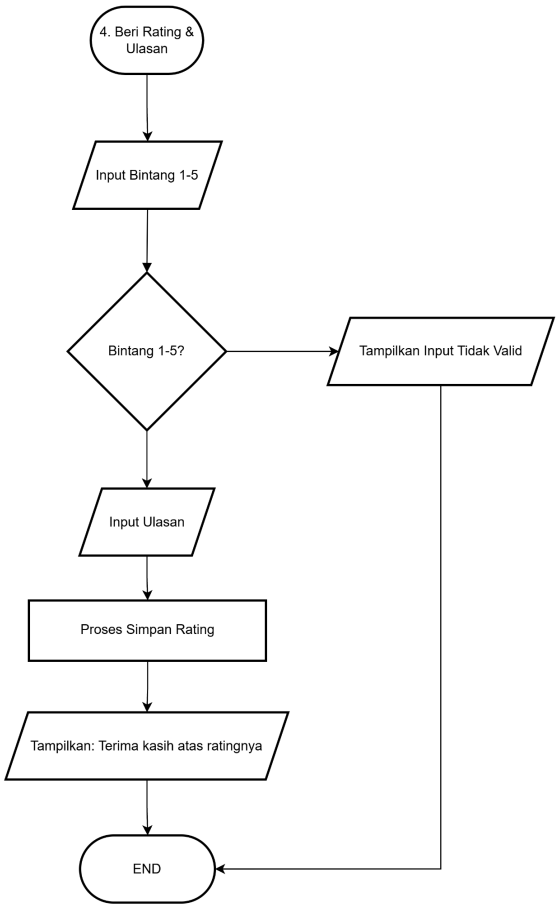

9. Urutkan Rating (Bintang - Ascending)
Mulai dari data rating kedua
Apakah sudah sampai di akhir data rating?
Belum
Ambil data rating saat ini sebagai kartu kunci
Bandingkan dengan data yang ada di sebelah kirinya
Apakah masih ada data di sebelah kiri dan rating kirinya lebih besar dari kunci?
Tidak
Taruh kartu kunci di posisi ini
Lanjut ke data rating berikutnya
Ya
Geser data kiri ke posisi sebelah kanannya
Mundur ke kiri lagi untuk membandingkan
Ya
Tampilkan semua rating yang sudah terurut dari bintang terkecil ke terbesar
END

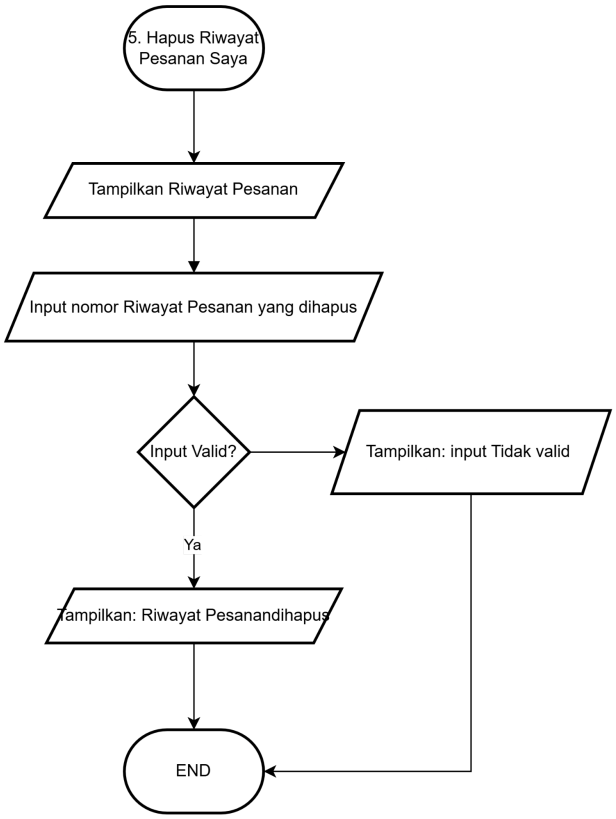

Menu Pengguna
Tampilkan Menu Pengguna
1. Info Detail Lapangan
2. Sewa Lapangan (Booking)
3. Riwayat Pembayaran
4. Beri Rating & Ulasan
5. Hapus Riwayat Pesan Saya
6. Hapus Rating Saya
7. Urutkan Lapangan (Nama - Ascending)
8. Urutkan Rating (Bintang - Ascending)
9. Urutkan Riwayat Saya (Tagihan - Descending)
10. Keluar
Input Pilihan Menu
Pilihan = 1?
Pilihan = 2?
Pilihan = 3?
Pilihan = 4?
Pilihan = 5?
Pilihan = 6?
Pilihan = 7?
Pilihan = 8?
Pilihan = 9?
Pilihan = 10?
Tampilkan: Pilihan Tidak Valid
1. Info Detail Lapangan
2. Sewa Lapangan (Booking)
3. Riwayat Pembayaran
4. Beri Rating & Ulasan
5. Hapus Riwayat Pesan Saya
6. Hapus Rating Saya
7. Urutkan Lapangan (Nama - Ascending)
8. Urutkan Rating (Bintang - Ascending)
9. Urutkan Riwayat Saya (Tagihan - Descending)
END

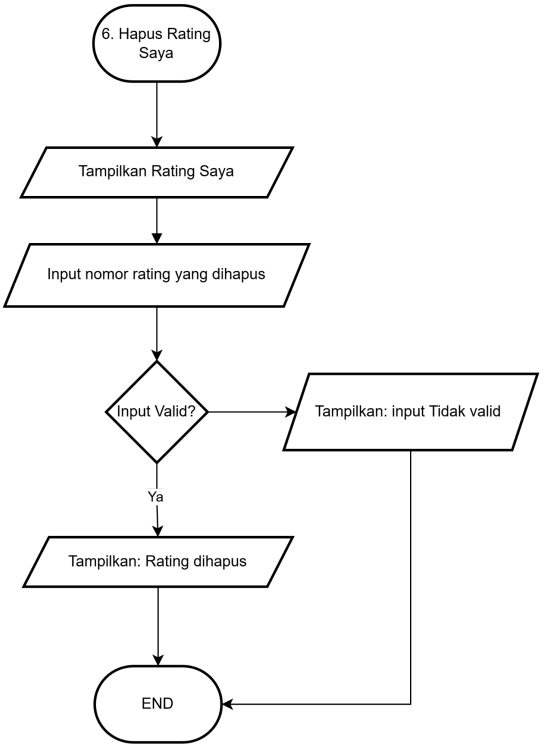

1. Info Lapangan
Tampilkan Info Lapangan
END

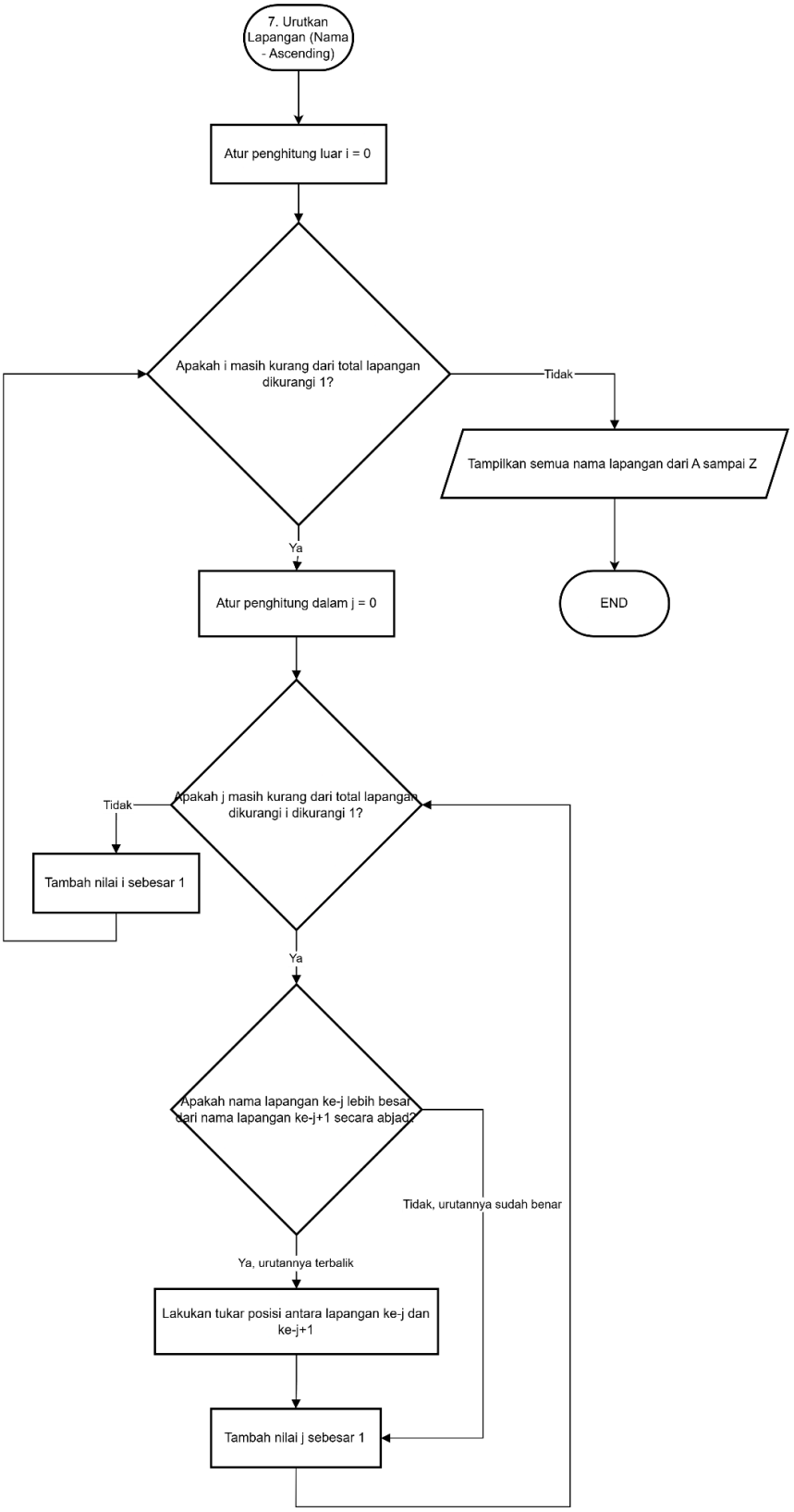

2. Sewa Lapangan (Booking)
Input Nama Pemesan
Input Tanggal
Input Jam
Input Durasi Main
Input valid?
Tidak
Tampilkan Input tidak valid
Ya
Hitung Total Tagihan
Tampilkan Total Tagihan
Input Uang Bayar
Uang cukup?
Tidak
Tampilkan Uang kurang
Hitung Kembalian
Tampilkan Berhasil + Kembalian
Update Kuota Lapangan dan Simpan Reservasi
END

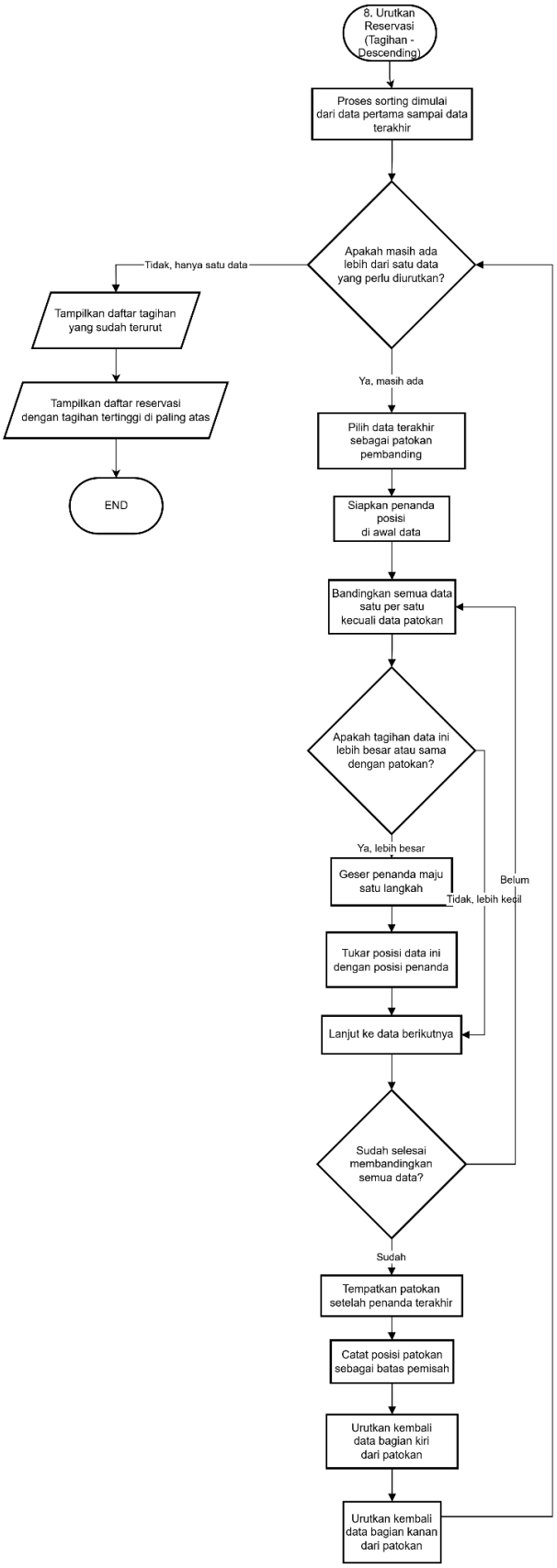

3. Riwayat Pembayaran
Ada riwayat?
Tidak ada Riwayat Pembayaran
Tampilkan Riwayat Pembayaran
END

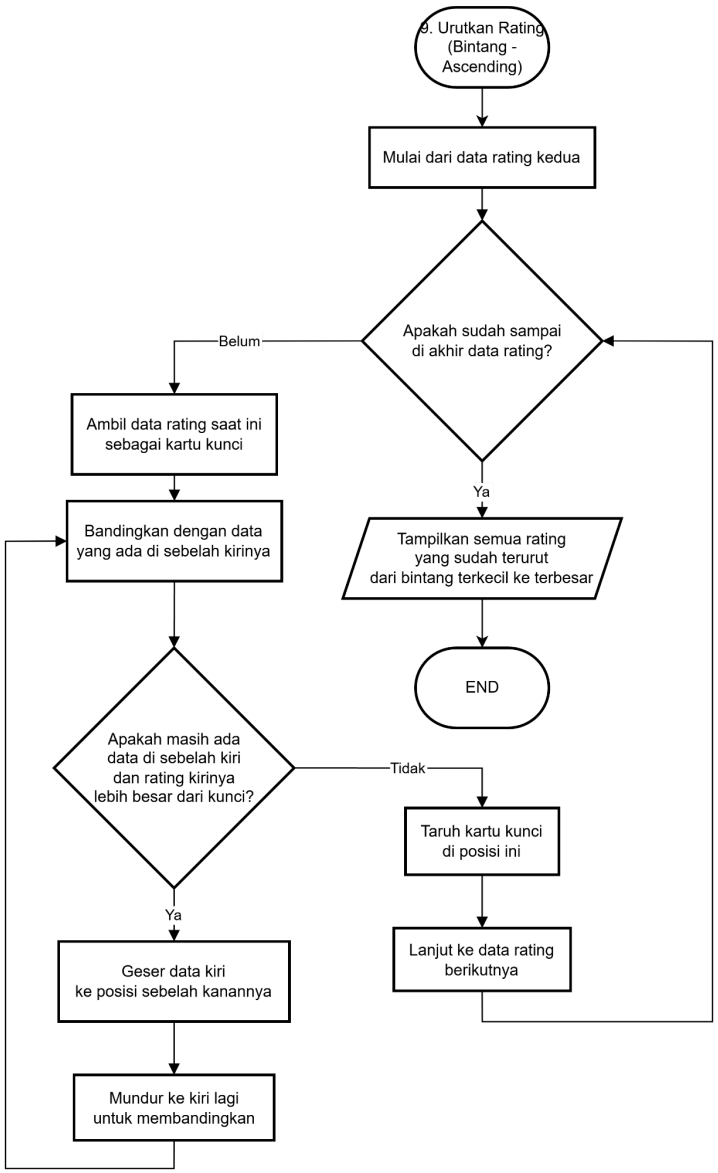

4. Beri Rating & Ulasan
Input Bintang 1-5
Bintang 1-5?
Tampilkan Input Tidak Valid
Input Ulasan
Proses Simpan Rating
Tampilkan: Terima kasih atas ratingnya
END


5. Hapus Riwayat Pesanan Saya
Tampilkan Riwayat Pesanan
Input nomor Riwayat Pesanan yang dihapus
Input Valid?
Tampilkan: input Tidak valid
Ya
Tampilkan: Riwayat Pesanandihapus
END


6. Hapus Rating Saya
Tampilkan Rating Saya
Input nomor rating yang dihapus
Input Valid?
Tampilkan: input Tidak valid
Ya
Tampilkan: Rating dihapus
END


7. Urutkan Lapangan (Nama - Ascending)
Atur penghitung luar i = 0
Apakah i masih kurang dari total lapangan dikurangi 1?
Tidak
Tampilkan semua nama lapangan dari A sampai Z
END
Ya
Atur penghitung dalam j = 0
Apakah j masih kurang dari total lapangan dikurangi i dikurangi 1?
Tidak
Tambah nilai i sebesar 1
Ya
Apakah nama lapangan ke-j lebih besar dari nama lapangan ke-j+1 secara abjad?
Tidak, urutannya sudah benar
Ya, urutannya terbalik
Lakukan tukar posisi antara lapangan ke-j dan ke-j+1
Tambah nilai j sebesar 1


8. Urutkan Reservasi (Tagihan - Descending)
Proses sorting dimulai dari data pertama sampai data terakhir
Apakah masih ada lebih dari satu data yang perlu diurutkan?
Tidak, hanya satu data
Tampilkan daftar tagihan yang sudah terurut
Tampilkan daftar reservasi dengan tagihan tertinggi di paling atas
END
Ya, masih ada
Pilih data terakhir sebagai patokan pembanding
Siapkan penanda posisi di awal data
Bandingkan semua data satu per satu kecuali data patokan
Apakah tagihan data ini lebih besar atau sama dengan patokan?
Ya, lebih besar
Geser penanda maju satu langkah
Tidak, lebih kecil
Belum
Tukar posisi data ini dengan posisi penanda
Lanjut ke data berikutnya
Sudah selesai membandingkan semua data?
Sudah
Tempatkan patokan setelah penanda terakhir
Catat posisi patokan sebagai batas pemisah
Urutkan kembali data bagian kiri dari patokan
Urutkan kembali data bagian kanan dari patokan



1.1 Deskripsi Flowchart

Flowchart sistem GOR Badminton Sanjaya menggambarkan alur kerja program mulai dari proses awal dijalankan hingga program selesai. Sistem dirancang menggunakan struktur percabangan dan perulangan untuk mengatur jalannya menu dan proses transaksi.

2. Deskripsi Singkat Program

Sistem Reservasi GOR Badminton Sanjaya dibuat secara khusus untuk mengotomatisasi proses penyewaan lapangan bulu tangkis. Tujuan utamanya adalah memudahkan pelanggan dalam mengecek ketersediaan lapangan, melakukan penyewaan, dan memberikan ulasan. Di sisi lain, sistem ini memberikan kemudahan bagi pihak pengelola untuk memantau data inventaris lapangan dan masukan pelanggan secara langsung.

3. Source Code

3.1 Bubble Sort (Sorting Huruf secara Ascending)

Source Code:

```
void bubble_sort_nama_lapangan(Lapangan* daftar_lapangan, int
jumlah_lapangan) {
    for (int i = 0; i < jumlah_lapangan - 1; i++) {
        for (int j = 0; j < jumlah_lapangan - i - 1; j++) {
            if ((*(daftar_lapangan + j)).nama > (*(daftar_lapangan + j +
1)).nama) {
                Lapangan temp = *(daftar_lapangan + j);
                *(daftar_lapangan + j) = *(daftar_lapangan + j + 1);
                *(daftar_lapangan + j + 1) = temp;
            }
        }
    }
    cout << "\n========================================" << endl;
    cout << "SORTING: Bubble Sort" << endl;
    cout << "DATA: Nama Lapangan" << endl;
    cout << "URUTAN: Ascending (A-Z)" << endl;
    cout << "========================================" << endl;
}
```

Penjelasan:

Program menggunakan algoritma Bubble Sort untuk mengurutkan nama lapangan secara ascending (A-Z). Metode ini bekerja dengan membandingkan dua elemen yang bersebelahan, kemudian menukar posisinya jika elemen pertama lebih besar dari elemen kedua secara abjad. Proses ini diulang secara berkelanjutan hingga tidak ada lagi pertukaran yang terjadi, yang menandakan data sudah terurut. Implementasi menggunakan pointer untuk mengakses array struct Lapangan.

Contoh Penggunaan:

```
// Sebelum sorting: ["Semen Beton", "Vinyl Premium", "Karpet
Standar"]
```

```
bubble_sort_nama_lapangan(daftar_lapangan, jumlah_lapangan);
// Setelah sorting: ["Karpet Standar", "Semen Beton", "Vinyl
Premium"]
```

3.2 Quick Sort (Sorting angka secara Descending)

Source Code:

```
int partition_tagihan_reservasi(Reservasi* daftar, int low, int high) {
    int pivot = (*(daftar + high)).totalTagihan;
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if ((*(daftar + j)).totalTagihan >= pivot) {
            i++;
            Reservasi temp = *(daftar + i);
            *(daftar + i) = *(daftar + j);
            *(daftar + j) = temp;
        }
    }
    Reservasi temp = *(daftar + i + 1);
    *(daftar + i + 1) = *(daftar + high);
    *(daftar + high) = temp;
    return i + 1;
}

void quick_sort_tagihan_reservasi(Reservasi* daftar, int low, int high)
{
    if (low < high) {
        int pi = partition_tagihan_reservasi(daftar, low, high);
        quick_sort_tagihan_reservasi(daftar, low, pi - 1);
        quick_sort_tagihan_reservasi(daftar, pi + 1, high);
    }
}
```

Penjelasan:

Program menggunakan algoritma Quick Sort untuk mengurutkan total tagihan reservasi secara descending (terbesar ke terkecil). Metode ini bekerja dengan memilih satu elemen sebagai pivot (patokan), kemudian melakukan partisi untuk membagi data menjadi dua bagian: elemen yang lebih besar atau sama dengan pivot diletakkan di sebelah kiri, sedangkan elemen yang lebih kecil di sebelah kanan. Proses ini dilakukan secara rekursif pada kedua bagian hingga seluruh data terurut. Quick Sort memiliki kompleksitas waktu rata-rata O(n log n) yang lebih efisien untuk data dalam jumlah besar.

Contoh Penggunaan:

```
// Sebelum sorting: [80000, 150000, 60000, 200000]
quick_sort_tagihan_reservasi(daftar_reservasi, 0, jumlah_reservasi - 1);
// Setelah sorting: [200000, 150000, 80000, 60000]
```

3.3 Insertion Sort (Sorting angka secara Ascending)

Source Code:

```
void insertion_sort_bintang_rating(Rating* daftar_rating, int
jumlah_rating) {
    for (int i = 1; i < jumlah_rating; i++) {
        Rating key = *(daftar_rating + i);
        int j = i - 1;
        while (j >= 0 && (*(daftar_rating + j)).bintang > key.bintang) {
            *(daftar_rating + j + 1) = *(daftar_rating + j);
            j--;
        }
        *(daftar_rating + j + 1) = key;
    }
    cout << "\n========================================" << endl;
    cout << "SORTING: Insertion Sort" << endl;
    cout << "DATA: Bintang Rating" << endl;
    cout << "URUTAN: Ascending (Terkecil-Terbesar)" << endl;
    cout << "========================================" << endl;
}
```

Penjelasan:

Program menggunakan algoritma Insertion Sort untuk mengurutkan bintang rating secara ascending (terkecil ke terbesar). Metode ini bekerja dengan mengambil satu elemen sebagai kunci (key), kemudian membandingkannya dengan elemen-elemen sebelumnya yang sudah terurut. Jika elemen sebelumnya lebih besar dari kunci, elemen tersebut digeser ke kanan. Proses ini berlanjut hingga ditemukan posisi yang tepat untuk menyisipkan kunci. Insertion Sort cocok untuk data yang sudah hampir terurut atau dalam jumlah kecil, dengan kompleksitas waktu $O(n^2)$ namun efisien untuk kasus terbaik $O(n)$.

Contoh Penggunaan:

```
// Sebelum sorting: [5, 3, 4, 2, 5]
insertion_sort_bintang_rating(daftar_rating, jumlah_rating);
// Setelah sorting: [2, 3, 4, 5, 5]
```

4. Hasil Output

```
[ MENU ADMIN ]
1. Status Lapangan
2. Lihat Daftar Rating
3. Hapus Ulasan (Rating)
4. Tambah Lapangan
5. Hapus Lapangan
6. Edit Detail Lapangan
7. Urutkan Lapangan (Nama - Ascending)
8. Urutkan Reservasi (Tagihan - Descending)
9. Urutkan Rating (Bintang - Ascending)
10. Keluar
Pilih: 7

========================================
SORTING: Bubble Sort
DATA: Nama Lapangan
URUTAN: Ascending (A-Z)
========================================

NO   NAMA               HARGA       BAHAN          KAPASITAS DESKRIPSI
-------------------------------------------------------------------------------------
1    Karpet Standar     Rp40000     Sintetis       2         Latihan Pro
2    Semen Beton        Rp25000     Semen Halus    2         Ekonomis
3    Vinyl Premium      Rp60000     Rubber Vinyl   1         Standar BWF
```

Gambar 4.1:admin dan user dapat mengecek Urutan Lapangan secara Ascending.

```
[ MENU ADMIN ]
1. Status Lapangan
2. Lihat Daftar Rating
3. Hapus Ulasan (Rating)
4. Tambah Lapangan
5. Hapus Lapangan
6. Edit Detail Lapangan
7. Urutkan Lapangan (Nama - Ascending)
8. Urutkan Reservasi (Tagihan - Descending)
9. Urutkan Rating (Bintang - Ascending)
10. Keluar
Pilih: 8

========================================
SORTING: Quick Sort
DATA: Total Tagihan Reservasi
URUTAN: Descending (Terbesar-Terkecil)
========================================

--- SEMUA RESERVASI (TERURUT) ---
- [Rara] Vinyl Premium | Rp120000
- [Denny] Karpet Standar | Rp80000
- [Dapa] Semen Beton | Rp50000
```

Gambar 4.2:admin dan user dapat mengecek Urutan Reservasi secara Descending.

```
[ MENU ADMIN ]
1. Status Lapangan
2. Lihat Daftar Rating
3. Hapus Ulasan (Rating)
4. Tambah Lapangan
5. Hapus Lapangan
6. Edit Detail Lapangan
7. Urutkan Lapangan (Nama - Ascending)
8. Urutkan Reservasi (Tagihan - Descending)
9. Urutkan Rating (Bintang - Ascending)
10. Keluar
Pilih: 9

========================================
SORTING: Insertion Sort
DATA: Bintang Rating
URUTAN: Ascending (Terkecil-Terbesar)
========================================

--- SEMUA RATING (TERURUT) ---
1. [Rara] * 2 - Sepi
2. [Dapa] * 4 - cukup oke
3. [Denny] * 5 - Lapangan Bersih dan Lampu Terang
```

Gambar 4.3:admin dan user dapat mengecek Urutan rating secara Ascending.

5. Langkah-langkah GIT

5.1 GIT Add

```
>git add .
```

git add .
Perintah git add digunakan untuk memasukkan perubahan file ke dalam staging area.
Staging area adalah tempat sementara yang digunakan untuk menyiapkan file sebelum dilakukan commit dan tanda titik "." berarti menambahkan semua file yang ada dalam folder proyek.

5.2 GIT Commit

```
D:\Praktikum Semester 2\APL\Praktikum-APL\POST-TEST\post-test-apl-5>git commit -m "posttest-5"
[main d56f824] posttest-5
 4 files changed, 534 insertions(+)
 create mode 100644 Praktikum-APL/POST-TEST/post-test-apl-5/GorApp.exe
 create mode 100644 Praktikum-APL/POST-TEST/post-test-apl-5/GorFungsi.cpp
 create mode 100644 Praktikum-APL/POST-TEST/post-test-apl-5/GorSistem.cpp
 create mode 100644 Praktikum-APL/POST-TEST/post-test-apl-5/GorStruk.h
```

git commit -m "posttest-5"

GIT Commit adalah proses menyimpan snapshot perubahan file ke dalam repository Git.
Setiap commit akan memiliki ID unik (hash), waktu commit, penulis commit, dan pesan commit.
Pesan commit (-m) digunakan untuk menjelaskan perubahan yang dilakukan.

5.3 GIT Push

```
D:\Praktikum Semester 2\APL\Praktikum-APL\POST-TEST\post-test-apl-5>git push origin main
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 8 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (9/9), 69.25 KiB | 5.77 MiB/s, done.
Total 9 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 1 local object.
To https://github.com/AxyL1a/Praktikum_APL.git
   3fdf5a4..d56f824  main -> main
```

git push origin main
Perintah ini digunakan untuk mengirim seluruh commit dari repository lokal ke repository GitHub.